

Introduction and learning objectives

This tutorial connects two parts of neural-network learning: computing gradients with backpropagation and using those gradients to update the parameters. We first work backwards through an MLP, then study what gradient descent can guarantee.

By the end of the tutorial, you should be able to:

- Choose a suitable loss for binary classification and compute error signals backwards through an MLP using the chain rule.
- Write each weight gradient as error signal $\times$ input, and combine the gradients over the training examples.
- Use Taylor's theorem to derive a step-size condition under which gradient descent decreases the loss.
- Derive and interpret the $O(1/T)$ bound on the smallest squared gradient seen in the first T iterations.

Optional extension: derive the corresponding SGD bound and explain the term that remains when the step size is fixed.

1 Backpropagation with Multilayer Perceptrons

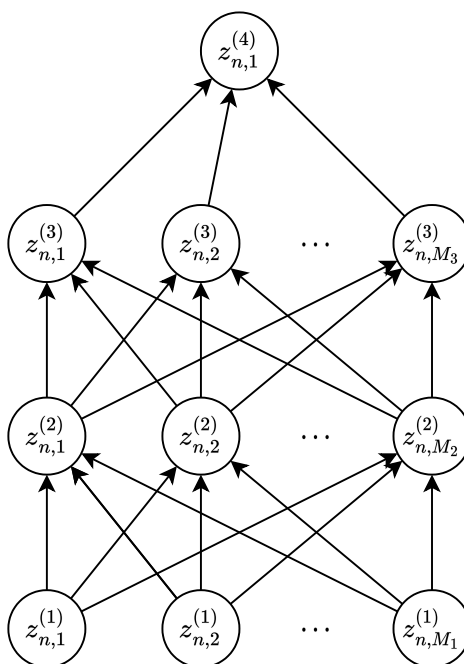


Figure 1: A neural network with two hidden layers.

Consider the network in Figure 1 with two hidden layers, input $\mathbf{x}$, and binary target $t \in \{0, 1\}$. Biases are omitted for simplicity.

Let $x_{n,i} = z_{n,i}^{(1)}$ be input component i for example n , and let $y_n = z_{n,1}^{(4)}$ be its output. The weight from $z_{n,i}^{(l)}$ to $z_{n,j}^{(l+1)}$ is $w_{j,i}^{(l)}$. We write θ for the set of all weights. Each unit computes

$$z_{n,j}^{(l+1)} = \sigma_{l+1} \left(a_{n,j}^{(l+1)} \right) \quad a_{n,j}^{(l+1)} = \sum_{i=1}^{M_l} w_{j,i}^{(l)} z_{n,i}^{(l)}$$

where $\sigma_2, \sigma_3, \sigma_4$ are the activation functions. In this problem, we let σ_2 and σ_3 be the ReLU function and σ_4 be the sigmoid function. For $x \neq 0$, the derivative of ReLU is $\mathbb{I}[x > 0]$. ReLU is not differentiable at zero; for this exercise, use the convention of assigning the value 1 there, so the backward calculation uses $\mathbb{I}[x \geq 0]$.

Problem 1. Write down the loss $\mathcal{L}(\theta; \mathcal{D}) = \sum_{n=1}^N \ell_n(\theta; \mathbf{x}_n, t_n)$ that is appropriate for our problem.

Problem 2. We will now practice manually performing backpropagation by calculating the gradients for each layer.

1. Write down the derivative $\delta_{n,1}^{(4)} = \frac{\partial \ell_n}{\partial a_{n,1}^{(4)}}$.
2. Write down the derivative $\frac{\partial \ell_n}{\partial w_{1,k}^{(3)}}$. You can leave your answer in terms of $\delta_{n,1}^{(4)}$.
3. Write down the derivative $\delta_{n,k}^{(3)} = \frac{\partial \ell_n}{\partial a_{n,k}^{(3)}}$. You can leave your answer in terms of $\delta_{n,1}^{(4)}$ and $w_{1,k}^{(3)}$.
4. Write down the derivative $\delta_{n,j}^{(2)} = \frac{\partial \ell_n}{\partial a_{n,j}^{(2)}}$. You can leave your answer in terms of $\delta_{n,k}^{(3)}$ and $w_{k,j}^{(2)}$.
5. Write down the derivative $\frac{\partial \mathcal{L}}{\partial w_{j,i}^{(1)}}$. You can leave your answer in terms of $\delta_{n,j}^{(2)}$.

Autograd. Hopefully through this simple but somewhat tedious exercise, you can appreciate how gradients are propagated backwards layer by layer through a neural network. This is the basis behind how modern deep learning libraries work, e.g. PyTorch's Autograd.

2 Gradient Descent

Recall that in gradient descent, our goal is to minimize a differentiable function $f(\mathbf{w})$. This process involves a series of iterative steps where we gradually adjust the parameter vector $\mathbf{w}$ using the following rule:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla_{\mathbf{w}_t} f(\mathbf{w}_t).$$

A natural question that arises from this algorithm is: does it converge? We shall try to provide an answer to this question in this problem. Throughout this question, we will consider scalar $\mathbf{w} \in \mathbb{R}$. Therefore, we will simplify our notation by using the non-bold version and represent $\mathbf{w} = w$.

Problem 1. Assume that the second derivative of f is bounded by some constant $L > 0$. In other words, we assume $f''(w) < L$. Recall that the Taylor's Theorem states that for any function $h : \mathbb{R} \rightarrow \mathbb{R}$ that satisfy some conditions (these conditions are satisfied in our context), there exist $\xi \in \mathbb{R}$ such that

$$h(x) = h(x_0) + h'(x_0)(x - x_0) + \frac{h''(\xi)}{2}(x - x_0)^2.$$

Using Taylor's Theorem or otherwise, show that

$$f(w_{t+1}) \leq f(w_t) - \alpha \left(1 - \frac{\alpha L}{2}\right) f'(w_t)^2.$$

Hint: Substitute w_{t+1} with update rule of gradient descent, then apply Taylor's Theorem.

Problem 2. Suppose the learning rate satisfies $0 < \alpha < 1/L$. Show that

$$f(w_t) - f(w_{t+1}) \geq \frac{\alpha f'(w_t)^2}{2}.$$

Problem 3. Let $f^* = \min_w f(w)$ be the optimal value of f . Show that

$$\frac{\alpha}{2} \sum_{t=0}^{T-1} f'(w_t)^2 \leq f(w_0) - f^*.$$

Problem 4. Show that the smallest squared gradient among the first T iterates has an $O(1/T)$ upper bound. In particular, show that

$$\min_{t \in \{0, \dots, T-1\}} f'(w_t)^2 \leq \frac{2(f(w_0) - f^*)}{\alpha T}.$$

Problem 5 (Optional). Let's repeat this derivation for stochastic gradient descent. At each step t , we update the weights using the formula $w_{t+1} = w_t - \alpha g_t$ where g_t is a *stochastic gradient* (i.e. it is a random variable) satisfying $\mathbb{E}[g_t | w_t] = f'(w_t)$ and $\mathbb{E}[g_t^2 | w_t] \leq \sigma^2$ for some $\sigma^2 > 0$. You should obtain:

$$\min_{t \in \{0, \dots, T-1\}} \mathbb{E}[f'(w_t)^2] \leq \frac{f(w_0) - f^*}{\alpha T} + \frac{\alpha \sigma^2 L}{2}.$$

The second term in this upper bound remains when the step size is fixed. Explain how the assumption on $\mathbb{E}[g_t^2 | w_t]$ gives rise to this term.